



preparatic 31

TEMA 126

LA SEGURIDAD EN EL NIVEL DE APLICACIÓN.
TIPOS DE ATAQUES Y PROTECCIÓN DE
SERVICIOS WEB, BASES DE DATOS E
INTERFACES DE USUARIO.

Versión	31.1
Fecha de actualización	23/02/2026

Resumen

ÍNDICE

1	La seguridad a nivel de aplicación	3
2	Seguridad en interfaces de usuario.....	3
2.1	Autenticación en aplicaciones o servicios	3
2.2	Autorización en el sistema	3
2.3	Gestión de excepciones.....	3
2.4	Validación de datos de entrada	3
2.5	Trazabilidad y auditorías	4
3	Seguridad en aplicaciones web	4
3.1	OWASP (Open Web Application Security Project)	4
3.2	Amenazas y ataques en aplicaciones web	4
3.2.1	A01:2025 Pérdida de control de acceso	4
3.2.2	A02:2025 Configuración de seguridad incorrecta	4
3.2.3	A03: 2025 Uso de componentes con vulnerabilidades conocidas	5
3.2.4	A04: 2025 Exposición de datos sensibles.....	5
3.2.5	A05: 2025 Inyección	5
3.2.6	A06:2025 Diseño inseguro	6
3.2.7	A07:2025 Pérdida de autenticación	6
3.2.8	A08:2025 Fallos de integridad de software/datos	6
3.2.9	A09:2025 Registro, monitorización y alerta insuficientes	7
3.2.10	A10:2025 Manejo incorrecto de condiciones excepcionales	7
3.2.11	OWASP cuadro resumen	8
3.2.12	Otros ataques.....	9
3.3	Metodologías de seguridad en aplicaciones web	9
3.4	La seguridad desde el punto de vista de la arquitectura de sistemas	9
4	Seguridad en servicios web.....	10
4.1	API REST	11
4.2	API SOAP	11
4.3	WS-Security.....	11
4.3.1	Seguridad a nivel de transporte (TLS) y a nivel de mensaje (WSS)	11
4.3.2	Tecnologías de autenticación, confidencialidad e integridad soportadas por WSS ..	12
4.3.3	Ejemplo de extensiones WSS	12
4.3.4	Otras extensiones para web services.....	13
4.3.5	Crítica a WSS y alternativas	14
4.4	SAML.....	14
4.4.1	Características generales	14
4.4.2	Mecanismos de funcionamiento	14

4.4.3	SAML y Web Service-Security.....	16
4.5	JSON Web Token	16
	Casos de uso	16
	Estructura.....	17
	Funcionamiento	17
5	Seguridad de las bases de datos	18

1 La seguridad a nivel de aplicación

La implementación de aplicaciones, cada vez más orientadas a sistemas abiertos, hace más necesario que, además de tener en cuenta cuestiones de seguridad a nivel de infraestructura o comunicaciones, sea necesario tener la seguridad en mente en el desarrollo de aplicaciones desde el comienzo de los proyectos.

Esta gestión debe ser multidisciplinar en cada una de las capas implicadas. En este tema resumimos las implicaciones en cuanto a la interfaz de usuario, la protección de los servicios web o la base de datos.

2 Seguridad en interfaces de usuario

La interfaz de usuario es el elemento de interacción entre el usuario y el sistema, y es preciso contemplar cuestiones de seguridad en su definición.

2.1 Autenticación en aplicaciones o servicios

Autenticación es el proceso por el que el usuario acredita su identidad a través de unas credenciales (algo que se tiene, como una tarjeta, algo que se sabe, como una contraseña, o algo que se es, como un elemento biométrico). Por lo general, se considera segura una combinación de dos de estos elementos, por ejemplo, una tarjeta criptográfica combinado con un elemento biométrico como la huella dactilar.

Las posibles amenazas en este elemento tienen que ver con el robo de identidad de una persona, es decir, identificarte ante un sistema como quien no eres, o en la suplantación de una sesión activa de otro usuario.

2.2 Autorización en el sistema

La autorización es el proceso por el que el sistema comprueba que un usuario tiene permiso para realizar la operación que ha solicitado. Las dos amenazas a evitar son la concesión de permisos superiores a los que debe tener un usuario, así como la denegación de permisos a un usuario que deba tenerlos. La gestión de autorizaciones se suele implementar mediante distintos mecanismos como las listas de control(ACL), o los modelos de control basados en roles (RBAC), o atributos (ABAC).

2.3 Gestión de excepciones

Las excepciones son comportamientos no esperados de los sistemas. En el caso de la interfaz de usuario, ante una excepción en la operativa, es conveniente que este hecho no proporcione al usuario información interna del sistema, que un usuario malintencionado pueda aprovechar para atacar al mismo.

2.4 Validación de datos de entrada

Un elemento crítico es la validación de datos proporcionados por el usuario a la interfaz de usuario. Es un elemento que suele obviarse en una implementación deficiente de un sistema y que puede provocar que, a través de la introducción de datos inválidos, un atacante interactúe con un sistema de forma no deseada, con objeto de obtener información a la que no está autorizado, suplantar a otro usuario, uso de inyección SQL, ataques de denegación de servicio, etc.

2.5 Trazabilidad y auditorías

Para la identificación de operativas incorrectas es conveniente que todas las capas de un sistema guarden información de excepciones o errores que permitan trazar su origen y, en su caso, corregir posibles defectos del software.

3 Seguridad en aplicaciones web

3.1 OWASP (Open Web Application Security Project)

La Fundación OWASP es un organismo sin ánimo de lucro formado por empresas, organizaciones educativas y particulares de todo el mundo. Es una comunidad de seguridad informática que trabaja para crear artículos, metodologías, documentación y herramientas que se liberan y pueden ser usadas gratuitamente. Publica y actualiza cada 3 años el proyecto **OWASP Top 10** que es un documento donde se recogen los diez riesgos de seguridad más importantes en aplicaciones Web. Mientras que **OWASP Top 10** indica que evitar, el **ASVS (Application Security Verification Standard)** es una lista de verificación detallada para auditar y asegurar que una aplicación Web es realmente robusta.

3.2 Amenazas y ataques en aplicaciones web

Los diez riesgos más críticos en Aplicaciones Web según OWASP Top 10 - 2025 son:

3.2.1 A01:2025 Pérdida de control de acceso

Si no se aplican correctamente las restricciones sobre lo que los usuarios pueden hacer, un atacante puede acceder de forma no autorizada a funcionalidades y datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar permisos de acceso, etc.

Cómo prevenir:

- La política debe ser denegar todo de forma predeterminada (excepción: recursos públicos).
- Se debe minimizar el uso de intercambio de recursos HTTP de origen cruzado (CORS).
- Se debe deshabilitar el listado de directorios del servidor web.
- Registrar errores de control de acceso que alerten a los administradores.
- Limitar la tasa de acceso a las APIs.
- Los tokens json (JWT) deben ser invalidados tras la finalización de la sesión. Gestionar expiración de tokens.
- Utilizar kits de herramientas o patrones con controles de acceso simples y declarativos.
- Incluir pruebas de control de acceso en las pruebas unitarias y de integración.

3.2.2 A02:2025 Configuración de seguridad incorrecta

Establecer una configuración por defecto o directamente una falta de configuración.

Cómo prevenir:

- Implementar procesos seguros de instalación sin funcionalidades innecesarias.
- Eliminar o no instalar frameworks y funcionalidades no utilizadas.
- Revisar y actualizar las configuraciones apropiadas como parte del proceso de gestión de parches.
- Entornos de desarrollo, preproducción y producción configurados de manera idéntica y con diferentes credenciales para cada entorno.
- Las aplicaciones deben tener arquitecturas segmentadas.
- Enviar directivas de seguridad a los clientes, por ejemplo, encabezados de seguridad.
- Un proceso automatizado para verificar la efectividad de las configuraciones y configuraciones en todos los entornos. Si es manualmente, mínimo una vez al año.

- Utilizar la federación de identidades, credenciales de corta duración o los mecanismos de acceso basados en roles (RBAC) o atributos (ABAC).

3.2.3 A03: 2025 Uso de componentes con vulnerabilidades conocidas

Los fallos de la cadena de suministro de software son deficiencias en el proceso de construcción, distribución o actualización de software. A menudo son causados por vulnerabilidades o cambios maliciosos en código de terceros, herramientas u otras dependencias en las que se basa el sistema.

Cómo prevenir:

- Gestionar centralizadamente la lista de materiales de software (SBOM) de todo el software.
- Reducir la superficie de ataque eliminando dependencias no utilizadas, características innecesarias, componentes, archivos y documentación.
- Monitorizar continuamente fuentes como CVE y NVD en búsqueda de vulnerabilidades.
- Utilizar herramientas de análisis automatizados.
- Suscribirse a alertas de seguridad.
- Obtener componentes únicamente de orígenes oficiales.
- Monitorizar, evaluar y aplicar actualizaciones. Gestión de cambios y configuraciones.
- Utilizar implementaciones por etapas o canarias para limitar la exposición.

3.2.4 A04: 2025 Exposición de datos sensibles

Defensas ante los ataques de exposición de datos sensibles, esto debe aplicarse no sólo al cifrado de los datos en tránsito sino también en reposo, así como una correcta gestión de los mismos por los usuarios.

Cómo prevenir.

- Clasificar y etiquetar los datos procesados, almacenados o transmitidos por una aplicación. Identificar qué datos son confidenciales por normativa o estrategia comercial.
- No almacenar datos sensibles innecesariamente.
- Utilizar implementaciones confiables de algoritmos criptográficos siempre que sea posible.
- Cifrar todos los datos confidenciales en reposo y en tránsito.
- Deshabilitar el almacenamiento en caché para respuestas con datos confidenciales.
- Almacenar contraseñas utilizando funciones de hashing.
- Utilizar el cifrado autenticado en lugar de solo el cifrado.
- Las claves deben generarse criptográficamente de forma aleatoria.
- Prepararse para la criptografía cuántica posterior.

3.2.5 A05: 2025 Inyección

Se produce al enviar datos no confiables a un intérprete (por ejemplo, un navegador, una base de datos, la línea de comandos). Los datos dañinos del atacante pueden engañar al intérprete para que acceda a información no autorizada o para que se ejecuten comandos involuntarios. Una aplicación es vulnerable a este tipo de ataques cuando los datos suministrados por el usuario no son validados y filtrados correctamente por la aplicación.

Algunas de las inyecciones más comunes son:

- Inyección SQL.
- Inyección SQL ciega (*Blind SQL*).
- Inyección en base de datos NoSQL.
- Inyección LDAP (& XPath).
- Inyección EL (Expression Language).
- En los LLM, existe la inyección en prompt.

Cómo prevenir:

- Separar los datos de los comandos y las consultas.

- Minimizar los permisos de la aplicación Web en la Base de Datos.
- Filtrar palabras SQL reservadas (y meta caracteres).
- Utilizar consultas parametrizadas y procedimientos almacenados.
- Realizar validaciones de entradas de datos en el servidor utilizando "listas blancas".
- Limitar mensajes de error.
- Utilizar firewalls de aplicación (WAF).
- Utilizar LIMIT y otros controles SQL.
- Pruebas automatizadas de todos los parámetros, encabezados, URL, cookies, JSON, SOAP y entradas de datos XML.
- Emplear de herramientas de prueba de seguridad de aplicaciones estáticas (SAST), dinámicas (DAST) e interactivas (IAST) en la canalización de CI/CD.

3.2.6 A06:2025 Diseño inseguro

Un diseño inseguro no puede arreglarse con una implementación perfecta, ya que, por definición, los controles de seguridad necesarios nunca se crearon para defenderse de ataques específicos.

Cómo prevenir:

- Establecer y utilizar un ciclo de vida de desarrollo seguro con los responsables de seguridad para ayudar a evaluar y diseñar controles relacionados con la seguridad y la privacidad
- Establecer y utilizar una biblioteca de patrones de diseño seguros.
- Utilizar el modelado de amenazas para partes críticas de la aplicación, como autenticación, control de acceso, lógica de negocio y flujos de claves.
- Formación integral a los usuarios sobre el uso seguro de las aplicaciones y sistemas.
- Escribir pruebas unitarias e integración para validar que todos los flujos críticos son resistentes al modelo de amenaza.
- Segregar las capas de nivel en el sistema y las capas de red, así como los clientes (tenants).

3.2.7 A07:2025 Pérdida de autenticación

Cuando un atacante es capaz de engañar a un sistema para que reconozca a un usuario inválido o incorrecto como legítimo, esta vulnerabilidad está presente.

Cómo prevenir:

- Implementar y aplique el uso de la autenticación multifactor (MFA).
- Fomentar y habilitar el uso de administradores de contraseñas.
- Implementar la política de seguridad para establecer duración de las contraseñas, longitud, complejidad, etc.
- Limitar o incrementar el tiempo de respuesta de cada intento fallido de inicio de sesión.
- Registrar todos los fallos de inicio de sesión.
- Utilizar un gestor de sesiones que genere un nuevo ID de sesión aleatorio.
- El Session-ID no debe incluirse en la URL.

3.2.8 A08:2025 Fallos de integridad de software/datos

Se refieren a código o infraestructura que no protege contra el código o datos inválidos o no confiables y se acaban tratando válidos y de confianza. El concepto de la lista de 2017 de deserialización insegura entraría en este grupo.

Cómo prevenir:

- Utilizar firmas digitales o mecanismos similares para verificar que el software o los datos provienen de la fuente esperada y no se ha modificado.
- Asegurarse de que las bibliotecas y dependencias, como npm o Maven, solo consumen repositorios de confianza.
- Garantizar que hay un proceso de revisión de los cambios de código y configuración.
- Asegurarse de la protección de la integridad del código en todo el pipeline de CI/CD.

- Aislar el código que realiza la deserialización. Registrar las excepciones y fallas en la deserialización. Monitorizar los procesos de deserialización.
- Restringir y monitorizar la conexión de red desde contenedores o servidores que utilizan funcionalidades de deserialización.

3.2.9 A09:2025 Registro, monitorización y alerta insuficientes

Sin registro y monitorización, los ataques y las brechas no se pueden detectar, y sin alertarlo es muy difícil responder de manera rápida y efectiva durante un incidente de seguridad.

Cómo prevenir:

- Asegurarse de que todos los fallos de inicio de sesión, control de acceso y validación de entrada del lado del servidor se puedan registrar para permitir un análisis forense.
- Asegurarse de que cada parte de su aplicación que contiene un control de seguridad esté registrada, ya sea que tenga éxito o falle y que los registros se generen en un formato que las soluciones de gestión de registros pueden consumir fácilmente.
- Codificar para proteger frente a inyecciones o ataques en el registro.
- Asegurarse de que todas las transacciones tengan una pista de auditoría con controles de integridad para evitar la manipulación o eliminación.
- DevSecOps y los equipos de seguridad establecerán monitorización y alerta efectivos, de modo que las actividades sospechosas sean detectadas y respondidas rápidamente por el equipo del Centro de Operaciones de Seguridad (SOC, en inglés).
- Agregue 'honeytokens' como trampas para atacantes en su aplicación.
- El análisis del comportamiento y el soporte de IA podrían ser opcionalmente una técnica adicional para soportar bajas tasas de falsos positivos para las alertas.
- Establecer o adoptar un plan de respuesta o recuperación de incidentes.

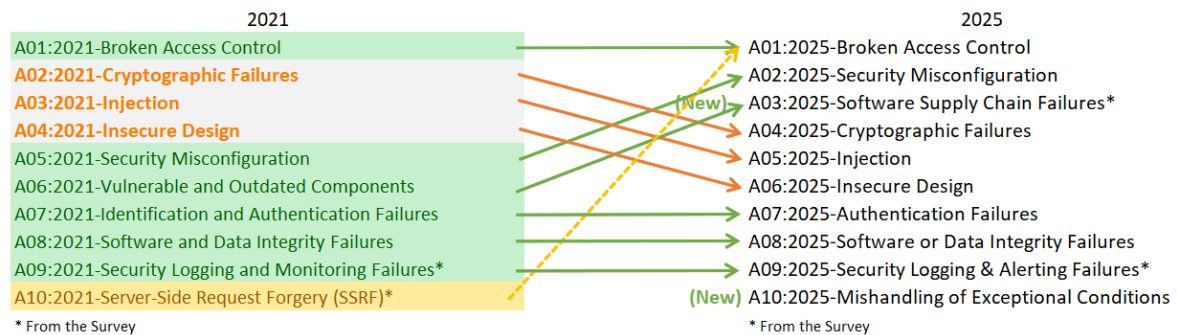
3.2.10 A10:2025 Manejo incorrecto de condiciones excepcionales

El mal manejo de las condiciones excepcionales en el software ocurre cuando los programas no previenen, detectan y responden a situaciones inusuales e impredecibles, lo que conduce a bloqueos, comportamientos inesperados y, a veces, vulnerabilidades. Esto puede implicar una o más de los siguientes tres fallos: la aplicación no impide que ocurra una situación inusual, no identifica la situación a medida que sucede y/o responde mal o no responde en absoluto a la situación después. Esta categoría busca asegurar que cuando un sistema falla, lo haga de manera controlada y segura, sin exponer la integridad ni la disponibilidad del sistema.

Cómo prevenir.

- Implementar un manejo de excepciones robusto: usar un único manejador global de excepciones para asegurar que todos los errores se procesen de la misma forma.
- Validación de datos estricta: implementar validación de entradas robusta para prevenir que datos malformados lleguen a capas lógicas vulnerables.
- Registros seguros sin información confidencial.
- Mensajes genéricos: mostrar errores amigables al usuario final y registrar los detalles técnicos solo en logs internos seguros.
- Diseño a prueba de fallos: configurar los sistemas para que, ante cualquier fallo crítico, la acción por defecto sea prohibir el acceso o la operación.
- Efectuar pruebas de estrés de los sistemas de información.

3.2.11 OWASP cuadro resumen



Hay dos nuevas categorías y una consolidación en el Top Ten para 2025

- **A01:2025 – Perdida de control de acceso** mantiene su posición en el #1. La falsificación de solicitud del lado del servidor (SSRF) se ha introducido en esta categoría.
- **A02:2025 - La configuración incorrecta** de seguridad al #2 en 2025. Esto no es sorprendente, ya que la ingeniería de software continúa aumentando la cantidad de comportamiento de una aplicación que se basa en configuraciones.
- **A03:2025 - Fallos de la cadena de suministro de software** es una expansión de A06:2021-Vulnerable y Obsoletos Componentes para incluir un alcance más amplio de compromisos que ocurren dentro o a través de todo el ecosistema de dependencias de software, sistemas de construcción e infraestructura de distribución.
- **A04:2025 - Los fallos criptográficos** caen dos lugares del #2 al #4 en el ranking. Nombrado anteriormente como Exposición de datos sensibles, el punto clave es la criptografía utilizada o la falta de esta, tanto en datos almacenados como en tránsito.
- **A05:2025 - La inyección** cae dos puntos del #3 al #5 en el ranking. La inyección incluye una variedad de problemas, desde secuencias de comandos entre sitios (alta frecuencia/bajo impacto) hasta vulnerabilidades de inyección SQL (baja frecuencia/alto impacto).
- **A06:2025 – Diseño Inseguro** se introdujo en 2021. Se han observado mejoras notables en la industria relacionadas con el modelado de amenazas y un mayor énfasis en el diseño seguro.
- **A07:2025 - Los fallos de autenticación** mantienen su posición en el #7 con un ligero cambio de nombre (anteriormente era “Identificación y fallas de autenticación”). Esta categoría sigue siendo importante, pero el mayor uso de marcos estandarizados para la autenticación parece estar teniendo efectos beneficiosos sobre los casos de fallos de autenticación.
- **A08:2025 - Los fallos de software o de integridad de datos.** Esta categoría se centra en la falta de mantenimiento de los límites de confianza y verificar la integridad del software, el código y los artefactos de datos a un nivel más bajo que los fallos de la cadena de suministro de software.
- **A09:2025 - Registro, monitorización y alerta.** Cambio de nombre (anteriormente Registro de seguridad y fallas de monitoreo) para enfatizar la importancia de la funcionalidad de alerta necesaria para inducir la acción apropiada en eventos de registro relevantes.
- **A10:2025 – Manejo incorrecto de condiciones excepcionales** es una nueva categoría para 2025. Esta categoría se centra en el manejo incorrecto de errores, errores lógicos, fallos abiertos derivados de condiciones anormales que los sistemas pueden encontrar.

3.2.12 Otros ataques

- **Desplazamientos por directorios (*path traversal*):** permiten a un usuario acceder sin control a ficheros y directorios que se almacenan fuera de la carpeta raíz de la aplicación Web.
- **Cross-Site Tracing (XST):** Es un tipo de *Cross Site Scripting* (XSS) que utiliza el método HTTP TRACE que se utiliza principalmente para fines de depuración repitiendo toda la información enviada por el cliente al servidor (petición “echo”).
- **Cross-Site Request Forgery (CSRF):** fuerza al navegador web de una víctima que está logado en algún servicio legítimo a enviar una petición a una aplicación web vulnerable con el fin de que la petición se procese en el nombre de la víctima.
- **Remote file inclusion (RFI) y Local File Inclusion (LFI):** RFI permite a un atacante ejecutar archivos remotos alojados en otros servidores. Esta vulnerabilidad existe solamente en páginas dinámicas en PHP. LFI es similar a RFI excepto en que en lugar de incluir ficheros remotos sólo se pueden incluir ficheros alojados en el mismo servidor víctima.
- **Ataques contra SSL/TLS:** Ver temas de seguridad en redes.
- **Fallos en el manejo de memoria:** Todavía hay muchos sistemas legacy en producción, nuevos sistemas de bajo nivel que requieren el uso de lenguajes con manejo directo y manual de memoria (como C, C++, Fortran, etc), y aplicaciones web que interactúan con mainframes, dispositivos IoT, firmware y otros sistemas que pueden verse obligados a administrar su propia memoria.
- **Confianza inapropiada en código generado por IA (vibe coding):** Existen problemas ligados a prácticas de desarrollo de software con código escrito y puesto en producción casi por completo sin la supervisión humana (programación por sensaciones/vibras).

3.3 Metodologías de seguridad en aplicaciones web

Una metodología de seguridad en aplicaciones web debe tener en cuenta lo siguiente:

- La seguridad en el ciclo de vida de desarrollo.
- Realizar pruebas de caja negra.
- Realizar pruebas de caja blanca.
- Implementar firewalls de nivel de aplicación (Web Application Firewalls, WAF).

OWASP ha desarrollado un marco de trabajo abierto denominado **SAMM** (*Software Assurance Maturity Model*) que pretende ayudar a las organizaciones en el proceso de formulación y aplicación de estrategias de seguridad en el software.

El CCN-CERT ha elaborado una [guía de seguridad para los entornos y las aplicaciones Web: CCN-STIC-812](#).

3.4 La seguridad desde el punto de vista de la arquitectura de sistemas

Seguridad en Arquitectura Monolítica

- Centralizada: la seguridad se gestiona en un solo lugar (autenticación, autorización, validación de datos).
- Menor superficie de ataque: al no haber servicios expuestos al exterior, la comunicación interna es más segura.
- Simple: más fácil de configurar, auditar y controlar al ser una sola unidad.
- Riesgo: si el perímetro de seguridad es comprometido, el atacante obtiene acceso a toda la aplicación.

Seguridad en Microservicios

- Granular (defensa en profundidad): cada microservicio puede tener sus propias políticas de seguridad y base de datos.
- Alta superficie de ataque: cada servicio expuesto aumenta los puntos de vulnerabilidad.
- Complejidad: requiere gestionar autenticación y autorización en múltiples componentes (API Gateways, servicios, redes).
- Resiliencia: si un servicio falla, la seguridad de los demás mantiene la integridad del sistema.
- Recomendación: uso obligado de protocolos de seguridad modernos (OAuth2, JWT) y, preferiblemente, un enfoque de Zero Trust.

4 Seguridad en servicios web

Las APIs (Interfaces de Programación de Aplicaciones) son conjuntos de reglas que permiten que distintas aplicaciones de software se comuniquen entre sí, facilitando el intercambio de datos y funcionalidades.

Principales riesgos en seguridad de API:

- Autorización de Nivel de Objeto Roto (BOLA): Es el riesgo más común. Ocurre cuando un atacante manipula el ID de un recurso (como `/api/usuarios/123`) para acceder a datos de otros usuarios sin permiso.
- Autenticación Quebrada: Fallos en la verificación de identidad que permiten a atacantes suplantar a usuarios legítimos o eludir el inicio de sesión.
- Autorización de Nivel de Propiedad de Objeto Roto: Exposición de campos sensibles en un objeto (como el sueldo en un perfil público) o permitir que un usuario cambie valores que no debería.
- Consumo de Recursos sin Restricciones: Falta de límites en el número de solicitudes (rate limiting), lo que facilita ataques de Denegación de Servicio (DoS) y ataques de fuerza bruta.
- Autorización de Nivel de Función Rota: Permite que usuarios con pocos privilegios ejecuten comandos de administrador (como borrar datos) simplemente cambiando el método HTTP o la URL.

Medidas que se pueden tomar para fortalecer la seguridad:

- Utilizar tokens. Configurar identidades confiables y controle el acceso a los servicios y a los recursos utilizando los tokens asignados a dichas identidades.
- Utilizar métodos de cifrado y firmas. Cifre sus datos mediante un método como TLS. Solicitar el uso de firmas para asegurar que solamente los usuarios adecuados descifren y modifiquen sus datos.
- Identificar vulnerabilidades, por ejemplo, utilizando un escáner web de vulnerabilidades como OWASP ZAP. Mantener actualizados sus elementos de API, sus controladores, redes y su sistema operativo. Mantenerse al tanto de cómo funciona todo en conjunto, e identifique las debilidades que se podrían utilizar para entrar en sus API. Utilizar analizadores de protocolos para detectar los problemas de seguridad y rastrear las pérdidas de datos.
- Utilizar cupos y límites. Establecer un cupo en la frecuencia con la que se puede recurrir a su API, y dé seguimiento a su historial de uso. Encontrar más solicitudes a una API puede indicar un abuso. También podría ser un error de programación, como una solicitud a la API en un bucle sin fin. Establecer reglas de limitación para proteger sus API de ataques de denegación de servicio y picos de uso.

- Utilizar una puerta de enlace de API. Las puertas de enlace de API funcionan como el principal punto de control para el tráfico de las API. Una buena puerta de enlace le permitirá autenticar el tráfico, así como controlar y analizar cómo se utilizan sus API

Existen numerosos tipos de APIs en la industria actualmente como SOAP, REST, GraphQL, gRPC y MQTT entre otros. En los siguientes apartados describiremos los dos primeros ya que tienen una mayor implantación, sin embargo, cada uno puede satisfacer un propósito diferente y se debe estudiar las características de cada uno de cara de decidir cuales implementar para cumplir con los requisitos de negocio.

4.1 API REST

La arquitectura de API de transferencia de estado representacional, o API REST, utiliza la transferencia de datos JSON principalmente y el protocolo de transferencia HTTP/S. Las API RESTful emplean solicitudes HTTP para crear (POST), actualizar (PUT), leer (GET) y eliminar (DELETE) datos.

La seguridad de las API REST, que carecen de unas disposiciones de seguridad propias, depende del diseño de la API. Los servicios de transmisión de datos, implementación e interacción con los clientes deben incorporar ciertas consideraciones de seguridad. La mayoría de las API RESTful usarán métodos de seguridad de la capa de transporte (como HTTPS) y autenticación basada en identificadores.

4.2 API SOAP

SOAP (protocolo simple de acceso a objetos) es un protocolo para intercambiar datos estructurados en la implementación de servicios web en redes de ordenadores. SOAP utiliza XML como formato de mensajes y puede transmitirse a través de distintos protocolos de niveles inferiores, como HTTP y SMTP. Las API SOAP suelen protegerse mediante una combinación de seguridad de la capa de transporte (como HTTPS) y en el nivel del mensaje (como XML, las firmas digitales y el cifrado). SOAP se adhiere a las especificaciones de los servicios web (WS) y a la extensión Web Services Security que veremos más en detalle a continuación.

4.3 WS-Security

WS-Security (WSS) es una extensión al estándar SOAP cuyo objetivo es proporcionar mecanismos de seguridad a los mensajes SOAP. En concreto, proporcionar autenticación, integridad y confidencialidad en el intercambio de estos. No desarrolla una nueva tecnología de seguridad sino que especifica cómo usar una serie de tecnologías y estándares de seguridad comúnmente aceptados para autenticación (por ejemplo, aserciones SAML o un certificado X.509 con una prueba de identidad), cifrado del mensaje (XML Encryption) y firma (XML DSignature).

De esta forma un proveedor de servicios web puede recibir un mensaje SOAP y mediante la lectura de las extensiones WSS de cabecera del mensaje SOAP identificar qué mecanismos ha escogido el emisor del mismo para autenticar al usuario final, cifrar el mensaje y firmarlo y de esta forma proceder a su validación. El objetivo final es ofrecer mecanismos de **interoperabilidad** que permitan la seguridad en los servicios web.

4.3.1 Seguridad a nivel de transporte (TLS) y a nivel de mensaje (WSS)

Diferencias entre los dos mecanismos:

TLS	WSS
Extremo a extremo de nivel de transporte (IP+puerto)	Extremo a extremo de nivel de aplicación.
Credenciales de autenticación de navegador y servidor únicos sin intermediarios.	Credenciales de autenticación de usuario final. Permite múltiples credenciales de autenticación.

No es flexible: se aplica a todo el payload de igual manera durante toda la sesión	Muy flexible: puede aplicarse sólo a una parte del mensaje y discriminar entre Requests y Responses
Se sitúa entre el nivel TCP y el de aplicación	La misma seguridad se puede aplicar en diferentes tecnologías de transporte.

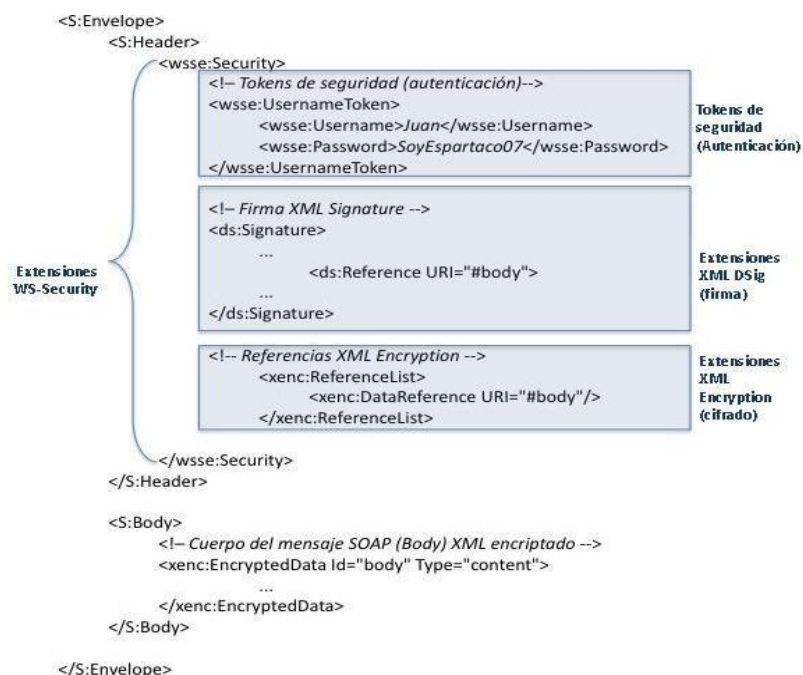
4.3.2 Tecnologías de autenticación, confidencialidad e integridad soportadas por WSS

Como se ha comentado, la norma WSS define extensiones en la cabecera SOAP para la autenticación, integridad y confidencialidad del mensaje. Estas extensiones, al final, están ligadas a las tecnologías de seguridad que se emplean. A continuación, se definen las tecnologías soportadas:

- **Integridad:** WSS emplea **XML Signature** (W3C) para la firma de los mensajes SOAP. Permite firmar cualquier documento XML incluido, naturalmente, mensajes SOAP en su totalidad o sólo partes de él. Para más información consúltase el tema sobre tecnologías de firma electrónica. Un mensaje SOAP puede llevar ninguna, una o múltiples firmas XML Signature.
- **Confidencialidad:** se basa en la especificación **XML-Encryption** (W3C) que al igual que ocurre con XML Signature permite encriptar parte o la totalidad del mensaje.
- **Autenticación:** WS-Security utiliza extensiones XML en la cabecera denominadas *tokens de seguridad*. El estándar WS-Security es muy flexible ya que define extensiones para muchos tipos de tokens posibles y también permite extender el modelo. Algunos ejemplos de tokens disponibles son **username/password**, certificados **X.509**, tickets **Kerberos**, aserciones **SAML**, tokens XrML, tokens XCBF o referencias a URIs. El uso de SAML permite también intercambiar aserciones de autorización para usuarios del servicio.

4.3.3 Ejemplo de extensiones WSS

Para su comprensión gráfica se muestra a continuación un esquema de un ejemplo de mensaje SOAP con extensión WS-Security. En este caso particular sólo habría una credencial pero podrían incluirse varios tokens de distintas tecnologías (aserciones SAML, Kerberos, certificados X.509 o incluso definidos por el propio usuario) o ninguno, de la misma forma que en el ejemplo sólo se muestra una firma, pero se podrían incluir varias o ninguna.



Ejemplo de mensaje SOAP con extensiones WSS

En cuanto a la implementación, la mayoría de los frameworks de desarrollo incluyen APIs para el desarrollo de servicios web con soporte a WSS. En concreto Java ofrece el API Java API for XML Web Services (JAX-WS) y en el entorno .Net el API para construir y consumir Web Services está integrado en Windows Communication Foundation.

4.3.4 Otras extensiones para web services

WS-Security se usa únicamente para autenticación, confidencialidad (cifrado) e integridad (firma). Existen otras extensiones SOAP **para el ámbito de la seguridad**. Su alcance es distinto del que ofrece WS-Security y por lo tanto todas son compatibles con este estándar. En concreto:

- **WS-Policy:** permite a un servicio web especificar sus requisitos de calidad de servicio y de seguridad para que puedan ser consultadas antes de invocar la interfaz WS. Las extensiones para especificar la tecnología aceptada de seguridad se denominan **WS-SecurityPolicy** y permiten definir qué tokens de autenticación se admiten, qué partes deben estar encriptadas, qué partes firmadas etc.
- **WS-Addressing:** proporciona mecanismos neutrales, independientes del protocolo de transporte, para direccionar mensajes y servicios web. Convierte los mensajes SOAP en unidades autónomas de comunicación
- **WS-Trust:** define cómo intercambiar tokens de seguridad con credenciales de acceso a servicios web de distintos dominios de confianza.
- **WS-Federation.** Una Federación es un conjunto de dominios de seguridad que han acordado compartir recursos de forma segura. Está soportado por las extensiones ofrecidas por WS-Security (seguridad en el mensaje SOAP), WS-Trust (intercambio de tokens de seguridad entre distintos dominios de seguridad), and WS-SecurityPolicy (publicidad de la política de seguridad).
- **WS-SecureConversation:** Extensión a WS-Security para el establecimiento de contextos de seguridad para el intercambio de una serie de mensajes SOAP durante toda la sesión que se establezca entre partes de un Web-Service (lo que se denomina una “conversación”). Se diferencia de WS-Security en que ésta norma protege cada uno de los mensajes SOAP de forma independiente, lo que introduce un elevado overhead, mientras que WS-SecureConversation cubre toda la conversación.



Pila de protocolos sobre la que se sustentan las extensiones Web Service

4.3.5 Crítica a WSS y alternativas

La principal crítica realizada a WS sobre SOAP y sus extensiones WSS es el overhead que introducen y su lentitud. Por ello en los últimos años se han popularizado las denominadas arquitecturas RESTful. RESTful es en realidad un concepto de arquitectura de servicios, pero comúnmente - aunque no sea muy acertado - se entiende este término como una alternativa a los servicios web basados en SOAP cuya principal característica es que ofrecen interfaces directas HTTP. Los servicios RESTful son más ligeros y su despliegue suele ser más rápido. Su seguridad suele estar garantizada mediante TLS.

4.4 SAML

SAML (*Security Assertion Markup Language*) es un lenguaje de marcado desarrollado por el consorcio OASIS para el intercambio de información de autenticación y autorización entre proveedores de servicios de identidad, proveedores de servicios y usuarios de los servicios. Permite el establecimiento de identidades portables, entendidas éstas como la extensión de las operaciones de autenticación y autorización de un dominio de seguridad a otro distinto con el que el primero mantiene una relación de confianza. La última versión es la 2.0 y data de 2005.

El uso más extendido de SAML es como mecanismo de Single Sign On y de federación de identidades pero su alcance va más allá de la mera autenticación permitiendo intercambiar atributos de los usuarios de los servicios con el fin de establecer perfiles para procesos de autorización. SAML se usa intensiva y extensivamente en la Administración General del Estado en servicios como Cl@ve y en su extensión europea STORK.

4.4.1 Características generales

- **Múltiples tecnologías de transporte:** Mecanismos más comunes: HTTP (Cl@ve) y SOAP.
- **Distribuido:** SAML no requiere una autoridad central.

4.4.2 Mecanismos de funcionamiento

- **Aserciones SAML.** Define esquemas XML para expresar lo que denomina “aserciones” (assertions), que son verificaciones constatadas sobre un usuario. Existen 3 tipos:
 - **Autenticaciones:** verificaciones de identidad.
 - **Autorizaciones:** las autoridades de autorización (normalmente las mismas que las de identificación) confirman o deniegan permisos de usuarios concretos ya identificados para realizar determinadas acciones.
 - **Atributos:** Expresan datos sobre una entidad (normalmente personas) que permiten a los proveedores de servicio aplicar su propia política de control de acceso o de servicio en función de la respuesta.
- **Roles.** SAML considera 3 roles:
 - **Principal:** se denomina así a la entidad que requiere ser autenticada; puede ser un usuario final u otro sistema.
 - **Proveedor de servicio (SP):** desea autenticar, autorizar o conocer algún dato de algún usuario o sistema (principal)
 - **Proveedor de identidad (IDP):** autentica, autoriza o envía datos de usuarios (principales).
- **Protocolos.**

SAML define esquemas XML para la implementación de protocolos sencillos Request/Response mediante los que se pueda solicitar aserciones y responderlas. Se trata de un esquema completamente separado del de aserciones.

Un requerimiento (request) contiene un claim mediante el que se solicita una aserción de autenticación, autorización o de atributos. La respuesta de la autoridad correspondiente contendrá la aserción resultante. Todo requerimiento debe contener el ID del sujeto en cuestión, que también se incluirá en la respuesta. Los requerimientos (Request) pueden ser, según se solicite, de autenticación, autorización o solicitud de atributos (*claim*).

La siguiente tabla muestra los protocolos y su función:

Protocolo	Función
Authentication Request Protocol	Solicitud de autenticación de un usuario
Single Logout Protocol	Define la forma de realizar un logout simultáneo de las sesiones asociadas a un usuario. El logout puede ser iniciado directamente por el usuario, o iniciado por un IDP o SP debido a la expiración por tiempo o también por decisión de un administrador
Assertion Query and Request Protocol	Solicitud de aserciones. La forma de Request de este protocolo permite solicitar a un IDP un Assertion haciendo referencia a su ID. La forma de Query define como solicitar Assertions en base al usuario y el tipo de Assertion
Name Identifier Management Protocol	Provee mecanismos para cambiar el valor o formato del identificador usado para referirse al usuario. El protocolo también define la forma de terminar una asociación de la identidad del usuario entre el IDP y el SP
Name Identifier Mapping Protocol	Provee un mecanismo para poder mapear un identificador de usuario usado entre un IDP y un SP de forma de obtener el utilizado entre el IDP y otro SP.
Artifact Resolution Protocol	Permite transportar los Assertions por referencia (a esta referencia se le denomina Artifact en el estándar) en vez de valor. Esto facilita el uso de SAML en un entorno web.

● Bindings

Un *binding* es especificación que indica cómo encapsular los mensajes Request/Response del protocolo SAML en el payload del mensaje de un mecanismo de transporte concreto.

SAML define numerosos bindings. Algunos de los más importantes son:

- Binding de SAML sobre SOAP (Binding SOAP): indica cómo insertar un mensaje SAML en el cuerpo (body) de un mensaje SOAP con indicaciones de cómo transportar a su vez el mensaje SOAP sobreHTTP.
- Binding de SAML sobre el método POST de HTTP. (Binding POST)
- Binding de SAML sobre mensajes HTTP de redirección (Binding Redirección).

● Perfiles SAML

Los perfiles SAML definen cómo combinar aserciones SAML, protocolos y bindings para poder cubrir determinados escenarios de uso y garantizar la interoperabilidad.

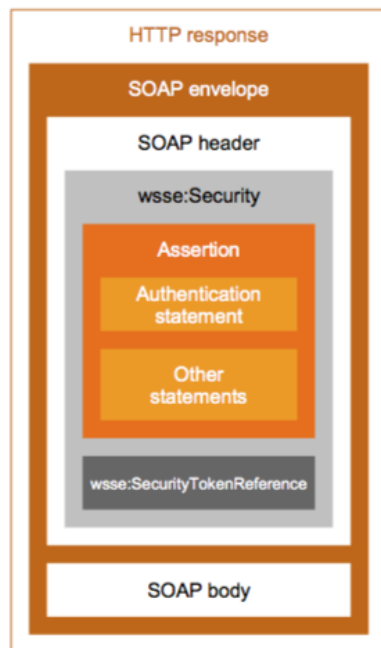
Cabe destacar:

- Perfil SSO para browsers (Web Browser SSO Profile). Pensado para ofrecer servicios de SSO a clientes con navegadores estándar.
- Perfil mejorado de cliente y proxy (Enhanced Client and Proxy - ECP). Caso especial en que se emplea un mediador de proveedores de servicios de identidad (caso de Cl@ve).

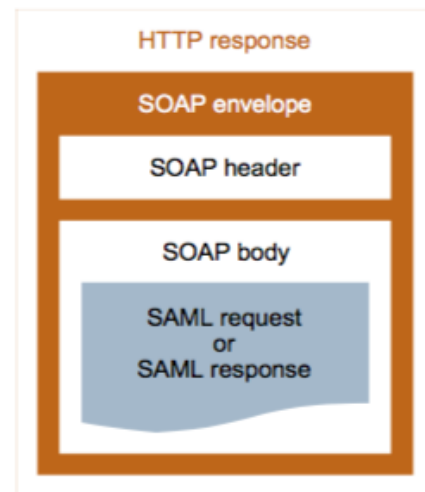
4.4.3 SAML y Web Service-Security

Como se ha comentado, WSS consiste en un conjunto de especificaciones para proporcionar confidencialidad, integridad y autenticación a mensajes SOAP. Para ello se especifica un tag (*wsse*) para la cabecera de los mensajes SOAP dentro del cual se coloca la información que proporciona la seguridad al mensaje.

Mientras que los mensajes de SAML sobre SOAP van en el cuerpo (*body*) de SOAP (ver binding SOAP en el apartado anterior) y forman parte de un protocolo Request/Response de autenticación, autorización o consulta de atributos, los mensajes SAML de la norma WSS son sólo aserciones que van en la cabecera (*header*) SOAP y tienen como objetivo participar de la seguridad del mensaje SOAP en sí. Normalmente son aserciones de autenticación que van firmadas mediante XML DSignture y que permiten establecer la identidad del emisor del mensaje SOAP.



Mensaje SOAP con extensiones WSS que incluye aserciones SAML como tokens de seguridad (en cabecera)



SAML sobre SOAP (en body)

4.5 JSON Web Token

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define una forma compacta y autónoma para transmitir de forma segura información entre partes como un objeto JSON. Esta información puede ser verificada y confiable porque está firmada digitalmente. Los JWT se pueden firmar usando un secreto (con el algoritmo HMAC) o un par de claves públicas/privadas usando RSA o ECDSA.

Los tokens firmados pueden verificar la integridad de las reclamaciones contenidas en él, mientras que los tokens cifrados ocultan esas reclamaciones a otras partes. Cuando los tokens se firman utilizando pares de claves públicas / privadas, la firma también certifica que solo la parte que tiene la clave privada es la que la firmó (no repudio).

Casos de uso

- **Autorización:** Este es el escenario más común para usar JWT. Una vez que el usuario ha iniciado sesión, cada solicitud posterior incluirá el JWT, permitiendo al usuario acceder a rutas, servicios y recursos que están permitidos con ese token. Single Sign On es una

característica que utiliza ampliamente JWT hoy en día debido a su pequeña sobrecarga y su capacidad para ser utilizado fácilmente en diferentes dominios.

- **Intercambio de información:** los tokens web JSON son una buena manera de transmitir información de forma segura entre las partes. Debido a que los JWT se pueden firmar, por ejemplo, utilizando pares de claves públicas / privadas, puede estar seguro de que los remitentes son quienes dicen que son. Además, como la firma se calcula utilizando el encabezado y la carga útil, también puede verificar que el contenido no se haya manipulado.

Estructura

- **Cabecera:** El encabezado típicamente consiste en dos partes: el tipo de token, que es JWT, y el algoritmo de firma que se está utilizando, como HMAC SHA256 o RSA.
- **Carga útil:** La segunda parte del token es la carga útil, que contiene las reclamaciones (claims). Las reclamaciones son declaraciones sobre una entidad (normalmente, el usuario) y datos adicionales. Hay tres tipos de reclamaciones: reclamaciones registradas (predefinidas), públicas y privadas.
- **Firma:** Para crear la parte de firma, debe tomar el encabezado codificado, la carga útil codificada, un secreto, el algoritmo especificado en el encabezado y firmarlo.

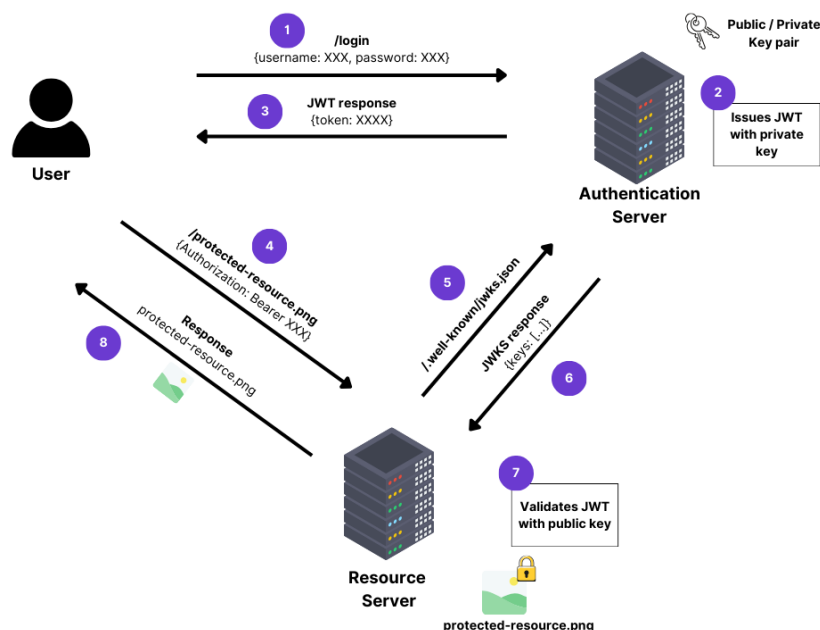
En su forma compacta, los tokens web JSON consisten en tres cadenas Base64-URL separadas por puntos (.), que son:

xxxxx.yyyyy.zzzzz
 <header(cabecera).Payload(Carga útil).Signature(firma)>

Funcionamiento

Los JWT se emiten y verifican. Cuando se emite, un servidor genera un nuevo JWT generalmente después de un proceso de inicio de sesión. Este paso implica añadir toda la información del usuario como reclamaciones JWT y firmar los datos con una clave privada. El cliente de usuario almacena este token y lo adjunta a cada solicitud de API futura.

Comúnmente, los JWT se envían como tokens de portador en el encabezado de autorización HTTP con el prefijo 'Bearer' (portador para que se le conceda acceso al recurso protegido). El servidor que recibe esta solicitud puede validar la firma del token con la clave pública y luego desempaquetar su contenido y autorizar la llamada.



<https://www.nblocks.dev/blog/authentication/introduction-to-json-web-tokens-jwt>

En esta figura el usuario quiere recuperar un recurso protegido del “Servidor de recursos”. Después de iniciar sesión y obtener un JWT desde el “Servidor de autenticación”, el usuario puede proporcionar este token al “Servidor de recursos” que tomará la clave pública del “Servidor de autenticación” y luego renderizará el recurso protegido.

Habitualmente, el login inicial se ha redireccionado desde el proxy inverso o API Gateway del sistema cuyos recursos se quieren acceder. Se debe tener en cuenta que con el término usuario puede tratarse de una persona (presentando sus credenciales como usuario y contraseña, certificado digital o otros medios autorizados) o un servicio web (identificándose por ejemplo mediante certificado cualificado de sitio web, acorde al Reglamento eIDAS).

5 Seguridad de las bases de datos

Otro de los elementos críticos de un sistema, son sus repositorios de datos, que generalmente se implementan a través de bases de datos relacionales, documentales o de otro tipo. Las vulnerabilidades más habituales que nos encontramos en las bases de datos son las siguientes:

- **Debilidad de credenciales:** Un usuario o una contraseña que puedan ser capturados por un atacante para acceder directamente al repositorio de datos impersonando un usuario válido.
- **Gestión de usuarios mediante grupos:** Una buena práctica es que los usuarios de una base de datos reciban sus privilegios a través de grupos, de forma que la seguridad se pueda manejar de forma colectiva.
- **Funciones no utilizadas:** Es conveniente, tanto en la base de datos, como en otro tipo de software asegurarnos que únicamente instalamos el software mínimo necesario para la operativa que tenemos. Una buena forma de evitar incorporar bugs a un sistema es no desplegar funcionalidades que no vamos a utilizar.
- **Mínimos privilegios:** Es otra buena práctica el definir para un grupo de usuarios los mínimos privilegios necesarios que precisa para operar en el sistema. No deben utilizarse más que cuando no exista otra alternativa para los usuarios administradores de la base de datos.
- **Elección y mantenimiento adecuados del SGBD:** Una elección del sistema de base de datos adecuada y un correcto y puntual mantenimiento del mismo pueden evitar problemas en la operación del mismo y riesgos de seguridad.
- **Inyección SQL:** Uno de los principales riesgos de uso de un gestor de base de datos, en particular cuando utilizamos bases de datos relacionales, es la inyección SQL. Una buena opción para protegerse del mismo es validar todas las entradas que se remiten al gestor de base de datos

